TrialMatch: Criterion-Level Clinical Trial Screening with a Fine-Tuned 1.5B Language Model, an Own KV Cache, Tiled Attention and Speculative Decoding

Muhammad Rafique
Roll no. 23i-0747 · Section CS-B
Generative AI, Assignment 2

Abstract—We fine-tune an open-weight decoder, Qwen2.5-1.5B-Instruct, to judge one patient note against one clinical-trial criterion at a time, returning a label from the dataset's four-label set and the ids of the note sentences that support it. Training uses QLoRA on a 6 GB laptop GPU and the expert annotations of the TrialGPT Criterion Annotations dataset, split by patient (37/8/8, seed 42). On the sealed test split the fine-tuned model reaches accuracy 0.709 and macro-F1 0.566, against 0.039 and 0.077 for the untrained model; evidence micro-F1 rises from 0.157 to 0.579. Around the model we implement three inference components from scratch: a cache-aware greedy decoder (identical tokens to full recomputation, up to 20.3× faster decoding at 2048 input tokens), tiled online-softmax attention (maximum fp32 error 1.8e-06, 317× less working memory at 8192 tokens) and greedy speculative decoding with a 0.5B draft (exactly the target's output, but 0.59–0.93× speed on this hardware, which we explain from measured draft and verify costs). A browser application exposes screening with evidence highlighting, a rule-based preliminary outcome and live benchmarks.

Index Terms—clinical trial matching, parameter-efficient fine-tuning, QLoRA, KV cache, online softmax, speculative decoding

I. Introduction

Recruiting patients for clinical trials requires checking a patient's record against every inclusion and exclusion criterion of a protocol. TrialGPT [1] showed that a large proprietary model can make these criterion-level judgements and cite the sentences it relied on. This project asks whether a small, open model, adapted on a laptop GPU, can learn the same task from expert annotations, and builds the inference machinery around it by hand.

The task is defined per criterion. Given a patient note split into numbered sentences, a criterion type (inclusion or exclusion) and the criterion text, the model returns JSON {"label": ..., "evidence_ids": [...]}. Inclusion criteria take included, not included, not enough information or not applicable; exclusion criteria take excluded, not excluded, not enough information or not applicable. The two "uncertain" labels are kept as first-class answers rather than collapsed into a binary decision. An explanation is built from the cited sentences, never generated, and a patient-level outcome is derived by an explicit rule (Section III-G) that is presented as a preliminary rule, not a prediction.

Contributions: (i) a reproducible patient-grouped split and evaluation protocol with explicit rules for empty evidence; (ii) a QLoRA sweep on a 6 GB GPU with selection on validation only; (iii) own implementations of a KV-cached decoder, tiled online-softmax attention and greedy speculative decoding, each verified for exactness and benchmarked; (iv) a FastAPI and Next.js application for unseen notes.

II. Related Work

Clinical eligibility. TrialGPT [1] matches patients to trials with GPT-4 in three stages and reports 87.3% criterion-level accuracy on the 1,015 expert-checked pairs we use; it relies on a paid API and is zero-shot. Wornow et al. [2] also prompt large hosted models zero-shot, for inclusion criteria only. We instead adapt a 1.5B open model and measure the zero-shot version of that same model as the baseline.

Architecture. The decoder follows the Transformer [3] with rotary position embeddings [4] and grouped-query attention [5]; Qwen2.5 [6] shares one tokenizer from 0.5B to 72B, which makes its 0.5B model a valid speculative draft for the 1.5B target.

Adaptation. LoRA [7] learns low-rank updates to frozen weights; QLoRA [8] keeps the frozen base in 4-bit NF4, which is what makes training possible in 6 GB.

Efficient inference. Pope et al. [9] separate compute-bound prefill from bandwidth-bound decode, which our benchmarks follow; PagedAttention [10] pages the KV cache for many concurrent requests, a design we considered and rejected for single-request use. Online softmax [11] and FlashAttention [12], [13] compute exact attention block by block; we implement the algorithm but not the fused kernel, which explains our speed results. Speculative decoding [14], [15] reduces to "accept while the argmaxes agree" under greedy decoding.

III. Materials and Methods

A. Dataset and split

We use the TrialGPT Criterion Annotations dataset [16] (1015 patient–criterion rows, 53 patients), pinned to revision 1cfcafde. Only expert-written columns are kept through an explicit allow-list; all GPT-4 output columns are dropped so they can never leak into training or evaluation. Notes arrive as numbered sentences, which we parse rather than re-split so evidence ids stay aligned with the annotations. Patients are shuffled with NumPy's generator seeded at 42 and split 37/8/8, so no patient appears in two splits (Table I). The split manifest with patient ids and file hashes is committed (results/split_manifest.json). The splits are strongly imbalanced: not excluded dominates exclusion rows, and the test split happens to contain 0 excluded rows, so that label cannot be scored on test.

B. Model and prompt

The target is Qwen2.5-1.5B-Instruct (28 layers, 12 query heads sharing 2 key/value heads, head size 128; results/model_candidates.json); the draft is Qwen2.5-0.5B-Instruct, whose tokenizer we verified identical (results/tokenizer_compat.json). The prompt (Fig. 1) is a system message plus the numbered sentences, the criterion type, the criterion and the meaning of its four labels; the answer is the compact JSON object. The longest training example is 636 tokens, so nothing is truncated at the 1,024-token limit (results/truncation_report.json). An unreadable answer is counted as a parse error, never guessed.

C. Fine-tuning

We train LoRA adapters on all attention and MLP projections (rank 16, α=32, dropout 0.05; 18.5 M trainable parameters) over an NF4 base with double quantisation and bf16 compute [8]. The loss is cross-entropy on the answer tokens only (prompt tokens masked with -100). We use batch size 1 with 8-step gradient accumulation, AdamW (lr 2×10^{-4}, no weight decay), clipping at 1.0, and 5% linear warm-up followed by cosine decay, for 5 epochs. To counter the label imbalance, one run weights each example's loss by the inverse square root of its (type, label) frequency, normalised to mean 1 and capped at 5. Four configurations (Table II) were run overnight; after every epoch the adapter is saved, validation loss is computed and the validation split is decoded and scored. The best run and epoch by validation macro-F1 is chosen; the test split is never used for selection. The chosen adapter is published on the Hugging Face Hub (Aduhere/trialmatch-qwen2.5-1.5b-qlora).

D. KV-cache decoding

Our runner executes the Qwen2 stack layer by layer, reusing only the model's weights and its linear, normalisation and MLP modules; positions, masks, grouped-query attention and the cache are our code. The cache holds preallocated per-layer key/value buffers of shape $[2, L_{\max}, 128]$ and a length counter; crop(n) rolls it back (used by speculative decoding). New tokens are rotated at their absolute positions $\ell..\ell{+}T{-}1$, where ℓ is the cache length, before their keys are stored, so cached keys never need re-rotation [4]. Query i may attend to keys $0..\ell{+}i$. Algorithm 1 gives the cached greedy loop; the reference decoder instead re-runs the whole sequence from an empty cache at every step. The cache costs $2\cdot28\cdot2\cdot128\cdot2$ bytes = 28 KB per token in fp16, growing linearly.

Algorithm 1 Cache-aware greedy decoding

Require: prompt ids $x_{1..P}$, max new tokens n, end token eos
$C \leftarrow$ empty cache; $z \leftarrow \text{Forward}(x_{1..P}, C)$ ▷ prefill
$y \leftarrow [\arg\max z_P]$
while $|y| < n$ and $y_{\text{last}} \neq$ eos do
  $z \leftarrow \text{Forward}(y_{\text{last}}, C)$ ▷ 1 token at position $|C|$
  append $\arg\max z$ to y ▷ C grew by one key/value per layer
end while
return y

Algorithm 2 Tiled causal attention, one query block

Require: Q_b (B_r rows), K, V, query offset q_0
$m \leftarrow -\infty$, $\ell \leftarrow 0$, $O \leftarrow 0$
for each key block (K_j, V_j) starting at k_0 do
  if $k_0 > q_0 + B_r - 1$ then break ▷ all future
  end if
  $S \leftarrow Q_b K_j^\top / \sqrt{d}$; mask future keys with $-\infty$
  $m' \leftarrow \max(m, \text{rowmax}(S))$; $P \leftarrow e^{S-m'}$; $\alpha \leftarrow e^{m-m'}$
  $\ell \leftarrow \alpha\ell + \text{rowsum}(P)$; $O \leftarrow \alpha O + PV_j$; $m \leftarrow m'$
end for
return O/ℓ ▷ rows with $\ell = 0$ return 0

E. Tiled attention with an online softmax

The reference attention materialises $S = QK^\top/\sqrt{d}$ in fp32, masks future keys with $-\infty$, subtracts each row's maximum before the exponential, and normalises; fully masked rows return zero instead of NaN. The tiled version (Algorithm 2) streams over key/value blocks and keeps, per query row, a running maximum m, normaliser ℓ and output accumulator O [11], [12]. Under causal masking, key blocks entirely in the future of a query block are skipped. Only one $B_r \times B_c$ tile of scores exists at a time, in pure PyTorch with no fused kernel.

F. Greedy speculative decoding

Algorithm 3 runs two of our runners with their own caches. The draft proposes γ tokens; the target scores the pending token plus all guesses in one forward pass; guesses are accepted while they equal the target's argmax, then the target's own token is emitted (a correction, or a bonus token when all γ matched). Both caches are cropped to the accepted length, so rejected guesses leave no trace and positions stay consistent. Every emitted token is the target's argmax given exactly its prefix, so the output equals plain greedy target decoding [14].

TABLE I
Patient-grouped split (seed 42) and label counts per split.

			Inclusion				Exclusion			
Split	Pat.	Rows	inc	not inc	NEI	NA	exc	not exc	NEI	NA
train	37	603	88	13	109	5	8	292	65	23
val	8	206	35	3	27	1	2	101	26	11
test	8	206	27	7	38	3	0	91	28	12

Source: results/split_manifest.json (dataset revision 1cfcafde94a1). NEI = not enough information, NA = not applicable.

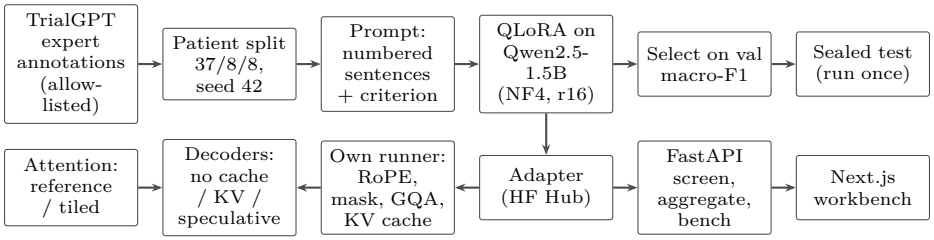


Fig. 1. System architecture. Top: data and training pipeline. Bottom: inference; the merged adapter feeds our own runner, whose attention function and decoders are the Task 3–5 implementations, served by the API and the browser app.

Algorithm 3 Greedy speculative round

Require: target cache C_t (covers all but pending token u), draft cache C_d

1: $d_{1..\gamma} \leftarrow$ greedy draft tokens from the draft model
2: $p_{0..\gamma} \leftarrow \arg\max \text{Forward}_t([u, d_1..d_\gamma], C_t)$ ▷ one pass
3: $k \leftarrow$ largest k with $d_i = p_{i-1}$ for all $i \le k$
4: emit $d_{1..k}$ and p_k; $\text{Crop}(C_t)$, $\text{Crop}(C_d)$ to the accepted length

G. Application

A FastAPI backend loads the fine-tuned model once (adapter merged into the fp16 base) and serves /screen, /aggregate, /benchmark, /attention-bench, /report and validation /examples; schema validation returns clear 422 errors, and GPU out-of-memory is caught. Unseen notes are split into sentences (a note already numbered "0. 1. …" is parsed as in training). The overall outcome follows a stated rule: any inclusion not included or exclusion excluded gives likely ineligible; otherwise any not enough information (or unreadable answer) gives needs review; otherwise likely eligible. The Next.js front end (Fig. 8) shows the note beside a case-report-form view in which every criterion displays all four possible answers with the model's ticked, the expert's answer circled on validation examples, and the cited sentences highlighted; a benchmark view runs the three decoders and the attention kernels live on the user's input. Fig. 2 shows one request end to end.

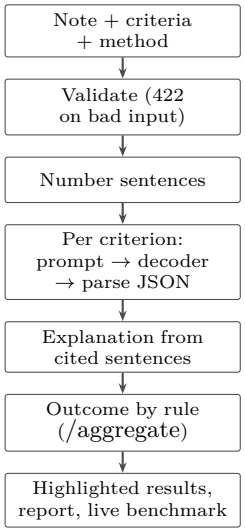


Fig. 2. Application workflow for one screening request.

IV. Experimental Setup and Metrics

All runs use one NVIDIA RTX 3050 6 GB laptop GPU, PyTorch 2.11 (CUDA 12.8), Transformers 5.22, PEFT 0.21 and bitsandbytes 0.50 on Windows 11. Evaluation decodes greedily with at most 64 new tokens. The evaluation runs of Tables III and II used the library's generate, which also applies the model's default repetition penalty of 1.1; Table VI shows this changes no label on validation and only the evidence lists. Our own decoders, used by the app and the benchmarks, are pure argmax.

Label metrics are accuracy, macro-F1 over labels present in the gold set, per-class precision/recall/F1 and confusion matrices, reported overall and per criterion type. Evidence is scored as sets of sentence ids: micro precision/recall/F1 over all rows, and macro F1 over rows; a row with empty gold and empty prediction is a correct abstention and is excluded from the macro average. Timings are wall-clock with torch.cuda.synchronize() around every phase, after one warm-up run, as means over 3–5 repeats; memory is torch.cuda.max_memory_allocated reset per run.

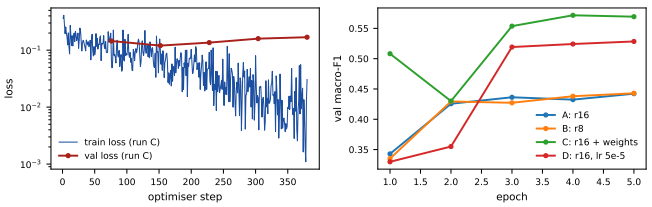


Fig. 3. Training curves. Left: per-step training loss and per-epoch validation loss for the selected run C. Right: validation macro-F1 per epoch for the four runs (source: results/train_*/).

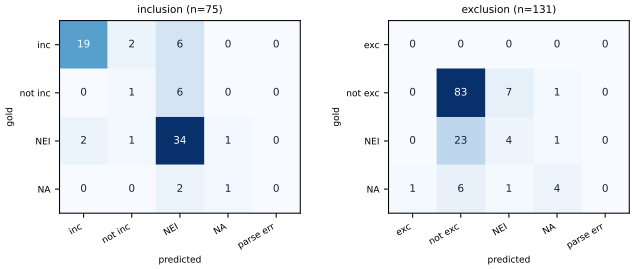


Fig. 4. Fine-tuned model confusion matrices on test, inclusion (left) and exclusion (right) criteria (source: results/finetuned_test/).

V. Results and Discussion

A. Fine-tuning and model selection

Table II and Fig. 3 summarise the sweep. Label weighting gave the largest gain: run C improved macro-F1 over the unweighted run A almost entirely through the rare exclusion labels, which it begins to predict. Rank 8 matched rank 16, so adapter capacity was not the bottleneck. Validation loss reached its minimum at epoch 2 for the 2e-4 runs and then rose while macro-F1 kept improving (Fig. 3): the model becomes more confident on frequent labels while still learning rare ones, so loss is not a good selection signal here. We selected run C: r16 + weights at epoch 4 (validation macro-F1 0.572 with the 4-bit in-training model; 0.527 re-scored with the fp16 base the app uses, the difference arising almost entirely from the two validation excluded rows).

B. Test results: before and after fine-tuning

Table III compares the untrained and fine-tuned models on the same 206 test criteria. The untrained model produces valid JSON but collapses to one label per criterion type and reads exclusion criteria with inverted polarity (its exclusion accuracy is zero). Fine-tuning raises accuracy from 0.039 to 0.709 and macro-F1 from 0.077 to 0.566, with no parse errors. Evidence micro-F1 rises from 0.157 to 0.579, and to 0.747 on rows where the expert cited at least one sentence. With 206 items the 95% interval on accuracy is roughly ±6 points, so differences of a few points against other runs are within noise. Per class (Table IV), frequent labels (included, not excluded) and inclusion not enough information score well; the rare not included and exclusion not enough information remain weak, which the confusion matrices in Fig. 4 make visible.

C. Error analysis

The fine-tuned model makes 60 label errors on test. The largest cells (Table V) reveal the main failure: 23 exclusion criteria labelled not excluded where the expert chose not enough information, i.e. the model treats a condition the note never mentions as absent. We traced ten cases to their sentence ids (results/error_analysis.md), including: absent evidence read as proof ("patients were moribund", a hypotensive patient labelled not excluded); clinical implications missed ("tachycardic" not recognised as heart rate ≥ 90); a negated criterion confused with a related procedure (arterial catheterisation read as central venous); a label contradicting the model's own correctly cited evidence (an SLE patient with malar rash and swollen joints labelled not included); temporal and frequency conditions ignored (snoring without the required three-month history accepted); confusion between not applicable and not enough information for sex- and age-dependent criteria; and over-citing background sentences when abstaining, the most common evidence-only error. Two cases show contradictory gold annotations: the experts themselves label "not mentioned" as not excluded for some criteria and not enough information for others, and label a pregnancy criterion for a 48-year-old man not excluded where a boy's equivalent is not applicable. Part of the residual error is therefore annotation inconsistency rather than model error; we report the gold labels unchanged.

D. KV cache

Cached decoding produced exactly the same tokens as both our full-recompute decoder and the library's uncached forward at 64/32, 256/64 and 1024/128 input/output tokens. Table VII and Fig. 5 show the benefit growing with prompt length: without the cache each new token costs about one prefill (875 ms at 2048 tokens), with the cache it stays near 43 ms, from 1.2× at 128 tokens to 20.3× at 2048. Cached decode is bound by reading the 3 GB of weights and launching small kernels, so its cost per token is almost independent of length. Peak memory barely differs because the weights dominate and the cache adds only 28 KB per token.

E. Tiled attention

Over 144 configurations (sequence lengths up to 2,048, head sizes 32–128, causal and non-causal, two block shapes) the tiled kernel matched the reference to at most 1.8e-06 in fp32 and 4.9e-04 in fp16, closer than PyTorch's fp16 SDPA (2.0e-03), because it accumulates in fp32 and rounds once. Replacing attention in all 28 layers of the model changed prefill logits by at most 0.13 and left 64 greedy tokens identical. Table VIII and Fig. 6 show the trade-off: at 8192 tokens the reference needs 9280 MB of working memory (beyond the 6 GB card, so

TABLE II
QLoRA sweep, best epoch of each run by validation macro-F1 (4-bit, in-training scoring).

Run	r	LR	Wts	Ep.	Val loss	Acc	Macro-F1	Excl F1	Evid. F1
A: r16	16	0.0002	no	5	0.157	0.738	0.442	0.445	0.548
B: r8	8	0.0002	no	5	0.157	0.743	0.443	0.427	0.555
C: r16 + weights	16	0.0002	yes	4	0.159	0.757	0.572	0.622	0.517
D: r16, lr 5e-5	16	5e-05	no	5	0.138	0.757	0.528	0.560	0.575

Source: results/train_*/summary.json. Wts = inverse-square-root label weighting.

TABLE III
Sealed test split, untrained (left of each pair) vs fine-tuned (right). Same 206 criteria, same prompt and decoding.

Subset	n	Accuracy		Macro-F1		Evid. micro-F1		Evid. macro-F1		Parse err.
overall	206	0.039	0.709	0.077	0.566	0.157	0.579	0.061	0.460	0 / 0
inclusion	75	0.107	0.733	0.099	0.541	0.217	0.626	0.065	0.559	0 / 0
exclusion	131	0.000	0.695	0.000	0.487	0.137	0.478	0.059	0.264	0 / 0

Source: results/baseline_test/metrics.json, results/finetuned_test/metrics.json. Macro-F1 averages labels present in the gold set; evidence F1 compares cited sentence ids.

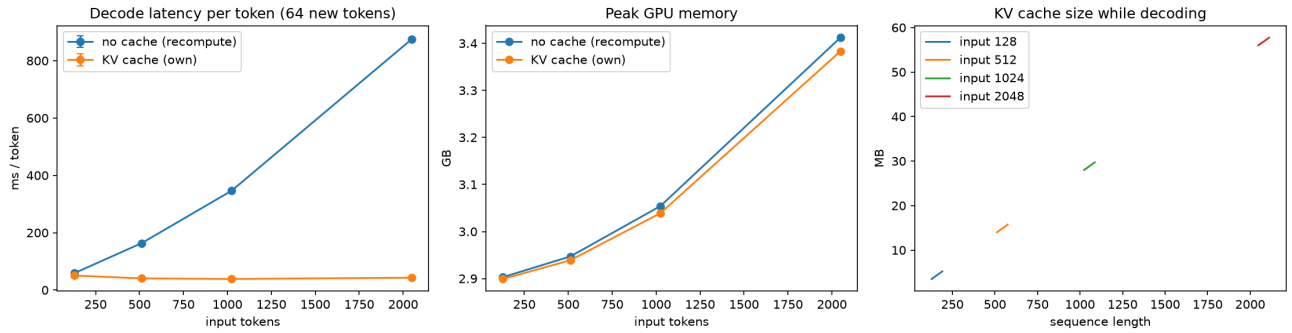


Fig. 5. KV-cache benchmark: decode latency per token, peak memory, and cache size while decoding (source: results/bench/kvcache.json).

TABLE IV
Per-class results on the test split (FT = fine-tuned).

Type	Label	Supp.	F1 base	F1 FT	P FT	R FT
inc.	included	27	0.312	0.792	0.905	0.704
inc.	not included	7	0.000	0.182	0.250	0.143
inc.	not enough information	38	0.000	0.791	0.708	0.895
inc.	not applicable	3	0.082	0.400	0.500	0.333
exc.	excluded	0	–	–	–	–
exc.	not excluded	91	0.000	0.818	0.741	0.912
exc.	not enough information	28	0.000	0.200	0.333	0.143
exc.	not applicable	12	0.000	0.444	0.667	0.333

Source: per_class in results/*_test/metrics.json. "–": label absent from the test gold set.

TABLE V
Largest error cells of the fine-tuned model on test.

Count	Gold	Predicted
23	not enough information	not excluded
7	not excluded	not enough information
6	not included	not enough information
6	not applicable	not excluded
6	included	not enough information

Source: confusion matrix in results/finetuned_test/metrics.json.

TABLE VI
Validation: generate's default repetition penalty (1.1) vs pure greedy (1.0).

Model	Rep. pen.	Acc	Macro-F1	Evid. F1
untrained	1.1	0.068	0.081	0.134
untrained	1.0	0.068	0.081	0.091
fine-tuned	1.1	0.738	0.527	0.507
fine-tuned	1.0	0.738	0.527	0.547

Source: results/{baseline,finetuned}_val{,_greedy}/metrics.json. No label changed in any of the 206 rows.

Windows spilled it to system memory) while the tiled version needs 29 MB, about 317× less, growing linearly rather than quadratically. Our tiling is slower than the fused kernels because each tile is a series of separate kernel launches from a Python loop with every intermediate round-tripping through GPU main memory; PyTorch's memory-efficient kernel performs the same tiling on-chip and obtains both the memory saving and the speed.

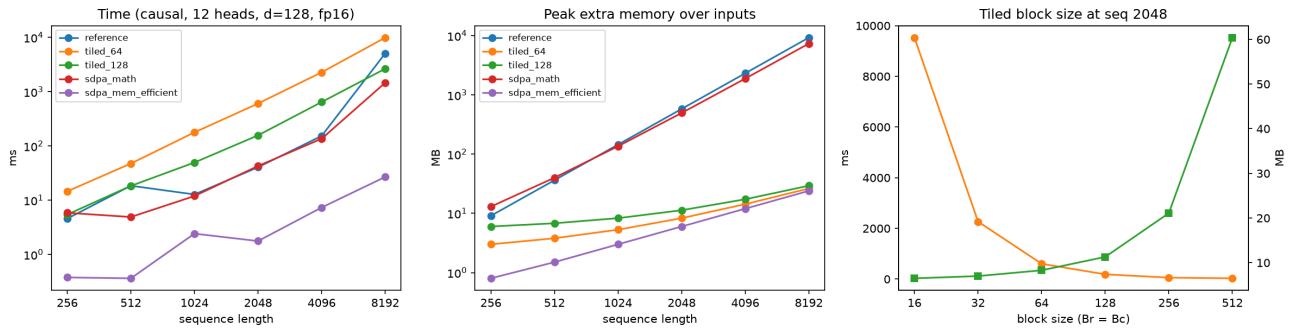


Fig. 6. Attention benchmark: time and extra memory versus sequence length, and the tiled block-size trade-off at 2,048 tokens (source: results/bench/attention.json).

TABLE VII
KV-cache benchmark, 64 new tokens, mean of 5 runs after warm-up.

Input	Prefill ms	ms/tok no cache	ms/tok cache	Speed-up	Peak GB	Cache MB
128	50	58.5	50.6	1.2×	2.90 / 2.90	5.2
512	150	163.6	40.6	4.0×	2.95 / 2.94	15.7
1024	327	346.4	38.7	9.0×	3.05 / 3.04	29.7
2048	808	874.7	43.1	20.3×	3.41 / 3.38	57.7

Source: results/bench/kvcache.json. Token equality with full recomputation at 64/32, 256/64, 1024/128 (input/output): all identical.

TABLE VIII
Attention time (ms) / extra memory (MB), causal, 12 heads, d=128, fp16.

Seq	Reference	Tiled (B=128)	SDPA math	SDPA mem-eff.
256	4.6 / 9	5.4 / 6	5.9 / 13	0.4 / 1
512	18.5 / 36	18.4 / 7	4.9 / 40	0.4 / 2
1024	12.8 / 145	49.6 / 8	11.9 / 136	2.4 / 3
2048	40.6 / 580	157.0 / 11	42.7 / 496	1.8 / 6
4096	153.0 / 2320	645.5 / 17	136.0 / 1888	7.3 / 12
8192	5086.9 / 9280	2651.6 / 29	1464.1 / 7360	26.9 / 24

Source: results/bench/attention.json (1 warm-up, 5 repeats). The flash SDPA backend is absent from the Windows PyTorch build.

F. Speculative decoding

Speculative output was identical to plain greedy decoding on every screening prompt at every γ, and to the library's assisted generation on 5/5 prompts; the single mismatch in a free-text workload was an exact fp16 tie (top-two logit gap 0.0000) broken differently by the one-token and multi-token kernels. Table IX and Fig. 7 show that the draft is accepted 0.59–0.87 of the time and that verification works as intended: checking γ+1 tokens costs about one target step (47 ms). The speed-up nevertheless stays below one (0.59–0.93×) because a draft step costs 40 ms, about 85% of a target step: in our launch-bound runner cost scales with the number of layers (24 vs 28), not with model width. A cost model with draft/target ratio c, speed-up $\approx$ tokens-per-pass$/(\gamma c + 1)$, reproduces the measured trend; the same acceptance rates with $c = 0.1$ would give about 2×. The fine-tuned target also lowers acceptance relative to the base target, since the untuned draft does not anticipate the compact JSON style.

TABLE IX
Speculative decoding, fine-tuned target, 20 screening prompts (plain greedy: 19.8 tok/s).

γ	Accept	Tok/pass	Draft ms	Verify ms	tok/s	Speed-up	Identical
1	0.869	1.74	40	47	18.4	0.93×	20/20
2	0.772	2.15	78	47	16.4	0.83×	20/20
3	0.810	2.67	117	47	16.2	0.82×	20/20
4	0.744	3.11	159	47	15.8	0.80×	20/20
6	0.650	3.46	229	50	13.6	0.69×	20/20
8	0.593	3.73	309	49	11.7	0.59×	20/20

Source: results/bench/speculative.json (1 warm-up, 3 repeats). Speed-up = plain cached greedy time / speculative time.

G. Application

Fig. 8 shows the screening view on a validation example: the selected criterion's cited sentence is highlighted in the note, each sentence lists the criteria that cite it, the model's answer is ticked among all four, the expert's answer is circled (with an inline note where they disagree), and the outcome is shown with the rule that produced it. The benchmark view runs all three decoders on the same input and reports identical answers with their measured timings.

VI. Limitations

The training set is small (603 rows from 37 patients) and stylistically narrow (emergency-department case vignettes), so notes written differently are out of distribution. The test split contains no excluded row, so that label is untested, and with 206 items the accuracy interval is about ±6 points. Annotation conventions for unmentioned conditions are inconsistent, which bounds achievable agreement. The evaluation decoding included the library's default repetition penalty (labels unaffected on validation, evidence lists slightly so). Our attention and speculative implementations demonstrate correctness and memory behaviour but not speed, because they are not fused kernels; speculative decoding did not pay off on this GPU. The patient-level outcome is a rule over predicted labels, not a validated prediction, and one wrong criterion can flip it. The system is an educational prototype and must not be used for clinical enrollment decisions.

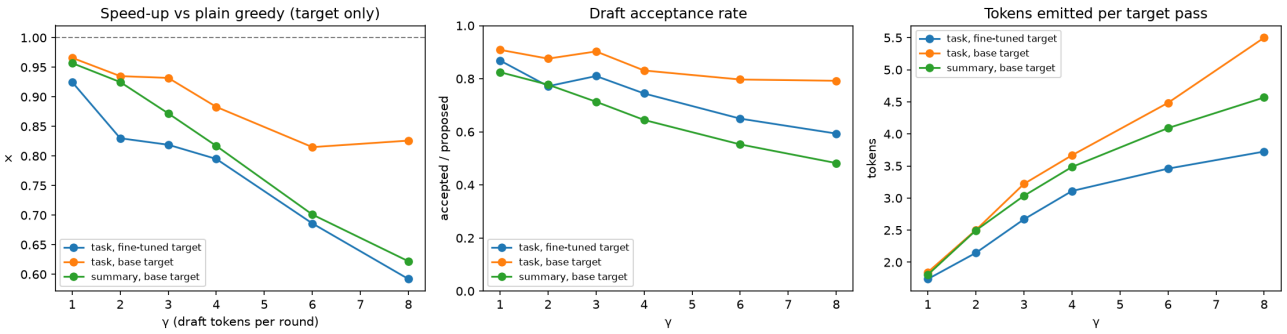


Fig. 7. Speculative decoding versus γ: speed-up over plain greedy, draft acceptance, and tokens emitted per target pass (source: results/bench/speculative.json).

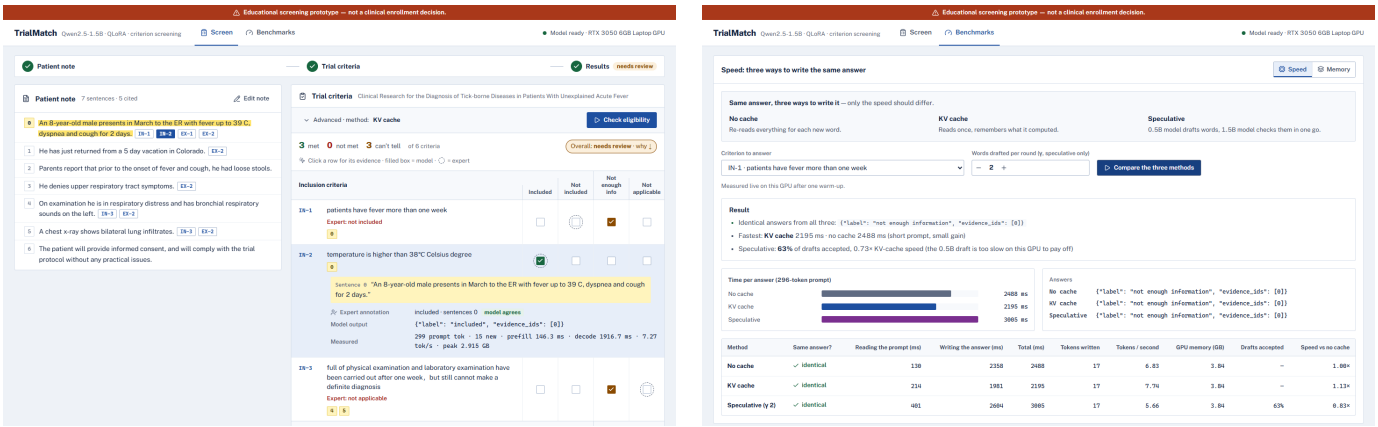


Fig. 8. Application: screening view with evidence highlighting and expert comparison (left); live decoder comparison on the same input (right). Captured on battery power, so the displayed timings are throttled.

VII. Conclusion

A 1.5B open model fine-tuned with QLoRA on a 6 GB laptop GPU learns criterion-level eligibility judgements with cited evidence, improving test accuracy from 0.039 to 0.709 and macro-F1 from 0.077 to 0.566. Our own KV cache, tiled attention and speculative decoder are exact by construction and verified token for token; the cache gives large decode speed-ups for long prompts, tiling gives large memory savings, and speculative decoding shows, with measurements, why a draft must be much cheaper than the target to help. Future work: more diverse training notes, a consistent convention for unmentioned conditions, and fused kernels for the attention and decoding paths.

References

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," Nature Communications, vol. 15, p. 9074, 2024.

[2] M. Wornow, A. Lozano, D. Dash, J. Jindal, K. W. Mahaffey, and N. H. Shah, "Zero-shot clinical trial patient matching with LLMs," 2024, arXiv:2402.05125.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[4] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," 2021, arXiv:2104.09864.

[5] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in Proc. EMNLP, 2023.

[6] Qwen Team, "Qwen2.5 technical report," 2024, arXiv:2412.15115.

[7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in Proc. ICLR, 2022.

[8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[9] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in Proc. MLSys, 2023.

[10] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in Proc. SOSP, 2023.

[11] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," 2018, arXiv:1805.02867.

[12] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in Advances in Neural Information Processing Systems (NeurIPS), 2022.

[13] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in Proc. ICLR, 2024.

[14] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in Proc. ICML, 2023.
[15] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," 2023, arXiv:2302.01318.
[16] NCBI, "TrialGPT criterion annotations," Hugging Face dataset ncbi/TrialGPT-Criterion-Annotations, revision 1cfcafde, 2024.

Appendix A
AI-use statement

An AI coding assistant (Anthropic's Claude, used through Claude Code) helped write code, tests, documentation and this report's first draft under the author's direction. Every step was planned on a task board, reviewed and approved by the author before being committed. Design aids (the impeccable and ui-ux-pro-max skills and Google Stitch) were used to explore the interface; designs proposing invented figures were not adopted. All experiments were run by the author on the stated hardware, and every number in this paper is generated from the result files in the repository by report/make_tables.py. No GPT-4 outputs from the dataset were used for training or evaluation.